Assignment 4 · Theory Answers

Yulin Zhang

Part 2 · Building Blocks of a Diffusion Model

1. Sinusoidal timestep embedding

Let the embedding dimension be an even integer d, with h = d/2 and maximum period P = 10,000. For frequency index j = 0, …, h − 1, compute $\omega_j = \exp(-\log(P)j/h) = P^{-j/h}$. Using zero-based coordinates and the cosine-then-sine convention used in this project,

$e_i(t)$
$\cos(tP^{-i/h})$ for $0 \le i < h$
$\sin(tP^{-(i-h)/h})$ for $h \le i < d$

2. Eight-dimensional embedding at timestep 1

Here h = 4. The frequencies are $10000^0 = 1$, $10000^{-1/4} = 0.1$, $10000^{-2/4} = 0.01$, and $10000^{-3/4} = 0.001$. Because t = 1, the angles have those same values. Applying cosine to the four angles, then sine to the four angles, gives:

```
[0.540302306, 0.995004165, 0.999950000, 0.999999500,
 0.841470985, 0.099833417, 0.009999833, 0.001000000]
```

3. Relation to Transformer positional encoding

Both methods map an integer to sine and cosine values at several frequencies. A Transformer uses positional encoding to give token representations information about their order in a sequence. A diffusion model uses a single timestep embedding per image to condition the U-Net on the noise level associated with that step. This embedding is projected and broadcast across spatial locations in each conditioned block.

4. Bottleneck resolution

Assuming each stride-2 convolution halves each spatial dimension, the sizes are 64 × 64 → 32 × 32 → 16 × 16 → 8 × 8. The bottleneck is **8 × 8**.

5. U-Net prediction and loss

In the noise-prediction formulation, sample $\varepsilon \sim N(0, I)$ and form $x_t = \sqrt{(\bar{\alpha}_t)}x_0 + \sqrt{(1 - \bar{\alpha}_t)}\varepsilon$, where $\bar{\alpha}_t = \prod_{s=0}^{t}(1 - \beta_s)$ for the implementation's zero-based timesteps. The U-Net predicts $\varepsilon_\theta(x_t, t)$, a tensor with the same shape as the image. The loss is the mean squared difference between that prediction and the exact noise sampled to create x_t:

$L = \text{mean}[(\varepsilon_\theta(x_t, t) - \varepsilon)^2]$.

Part 3 · Building Blocks of an Energy Model

6. Basic gradient calculations

a. For $y = x^2 + 3x$, $dy/dx = 2x + 3$. At $x = 2$, this is $2(2) + 3 = 7$. The code prints `x.grad = tensor([7.])`.

b. With `requires_grad=False`, the operations in this example produce `y.requires_grad == False`. Calling `y.backward()` raises a `RuntimeError` because there is no gradient graph for `y`. The gradient on `x` stays `None`.

c. The default is `requires_grad=False`. See torch.tensor.

7. Introduce weights

a. The original code prints `w.grad = None` because `w` was created with the default `requires_grad=False`. PyTorch still computes the gradient for `x`, which is 7.

b. For $y = w_0x^2 + w_1x$, the gradient with respect to w is $[x^2, x] = [4, 2]$ at $x = 2$.

```
import torch

x = torch.tensor([2.0])
w = torch.tensor([1.0, 3.0], requires_grad=True)
y = w[0] * x**2 + w[1] * x
y.backward()
print("w.grad =", w.grad)
# w.grad = tensor([4., 2.])
```

c. Omitting the flag gives `requires_grad=False` for the newly constructed tensor. See torch.tensor.

8. Breaking the graph

`y.detach()` removes the connection to the graph that contains `x`. The detached tensor and the resulting `w` do not require gradients in this example, so backward raises a `RuntimeError`. Assign `z = y` to keep the connection.

```
import torch

x = torch.tensor([1.0], requires_grad=True)
y = x * 3
z = y
w = z * 2
w.backward()
print("x.grad =", x.grad)
# x.grad = tensor([6.])
```

The chain rule gives dw/dx = 2 × 3 = 6. See Tensor.detach.

9. Gradient accumulation

The first backward call stores 2. The second adds 3 to the existing gradient, so the original example prints 2 and then 5. Set `x.grad = None` before the second call when you want its gradient independently. In training, the usual equivalent is `optimizer.zero_grad(set_to_none=True)` before the next backward pass.

```
import torch

x = torch.tensor([1.0], requires_grad=True)
(x * 2).backward()
print("First gradient:", x.grad)  # tensor([2.])
x.grad = None
(x * 3).backward()
print("Second gradient:", x.grad)  # tensor([3.])
```

See Tensor.backward.